PRACTICA 3

Transformación del Modelo Entidad-Relación Extendido al Modelo Relacional

Garcia Peñalva Saul
Hernández Martínez Clara Aide
Olarte Tomas Kevin Saul
3BV1
Bases de Datos

Contenido

Ejercicio 1: Análisis del Modelo EER Original 2
1.1 Revisión del Modelo de la Práctica 2 2
1.2 Preparación para la Transformación 2
Identificación de Nuevos Requisitos 3
Ejercicio 2: Reglas de Transformación Básicas 5
2.1 Transformación de Entidades Fuertes 5
2.2 Transformación de Atributos Compuestos 6
2.3 Transformación de Atributos Multivaluados 6
2.1 Cardinalidades Mínima y Máxima 7
2.2 Entidades Débiles 8
2.3 Herencia: Subtipos y Supertipos 9
2.4 Otros Conceptos 11
Ejercicio 3: Transformación de Relaciones 12
3.3 Relaciones Uno a Uno (1:1) 13
3.1 Diagrama en Notación Peter Chen 14
3.2 Diagrama en Notación Crow's Feet 14
Ejercicio 4: Transformación de Entidades Débiles 15
4.1 Explicación Detallada del Diseño 16
Ejercicio 5: Transformación de Jerarquías de Generalización/Especialización 17
..... 0
ANEXO 0
Ejercicio 6: Esquema Relacional Completo 0
Ejercicio 7: Validación y Verificación del Modelo 8
Bibliografía 11
ANEXO 11

Ejercicio 1: Análisis del Modelo EER Original

1.1 Revisión del Modelo de la Práctica 2

• Entidades fuertes

- **Cliente:** Atributos: id_cliente (PK), nombre (compuesto), RFC, correo, telefono, dirección (compuesto).
- **Usuario:** Atributos: id_usuario (PK), nombre (compuesto), contraseña, correo, teléfono (multivaluado), dirección (compuesto), estado.
- **Venta:** Atributos: id_venta (PK), fecha, total.
- **Proveedor:** Atributos: id_proveedor (PK), nombre_empresa, correo, tiempo_entrega, teléfono (multivaluado).
- **Categoría:** Atributos: id_categoria (PK), nombre.
- **Producto:** Atributos: id_producto (PK), nombre, precio_unitario, precio_caja, cantidad, fecha_ingreso, fecha_caducidad.

1.2 Preparación para la Transformación

Antes de iniciar la transformación, identifique:

• ¿Qué elementos requerirán tablas adicionales?

• ¿Qué atributos se convertirán en claves foráneas?

• ¿Qué decisiones de diseño deberá tomar en las jerarquías de herencia?

En la práctica anterior se desarrolló un **modelo Entidad-Relación (E-R)** para representar la operación de una tienda de abarrotes. Dicho modelo permitió identificar las entidades principales del sistema, tales como **Clientes, Usuarios, Productos, Categorías, Proveedores y Ventas**, así como las relaciones existentes entre ellas.

El modelo inicial cumplía con el objetivo de representar de manera general la estructura de la información necesaria para registrar ventas, administrar el inventario de productos, mantener información de los proveedores y registrar a los usuarios que operan el sistema. Sin embargo, al tratarse de un modelo E-R, presenta ciertas

limitaciones en cuanto a la representación de reglas más específicas y otras dependencias.

Entre las principales limitaciones identificadas se encuentran:

- **Falta de especificación de cardinalidades mínimas y máximas**, ya que el modelo inicial únicamente considera cardinalidades generales (1:1, 1:N o N:M) sin detallar restricciones más precisas de participación.
- **Ausencia de entidades dependientes**, ya que algunas estructuras de información, como los elementos que componen una venta, dependen directamente de la existencia de otra entidad principal.
- **No se distingue formalmente entre tipos de usuarios**, a pesar de que el sistema maneja dos roles diferentes: **administradores y empleados**, los cuales poseen características y privilegios distintos.
- **Algunas relaciones complejas no están representadas con suficiente detalle**, particularmente aquellas relacionadas con las transacciones de venta, donde intervienen múltiples entidades.

Debido a estas limitaciones, se planteará el **Modelo Entidad-Relación Extendido (EER)**, el cual permite representar de mejor manera la estructura y comportamiento de los datos dentro del sistema.

Identificación de Nuevos Requisitos

Restricciones de Cardinalidad Específicas

El funcionamiento del sistema implica diversas reglas de negocio que requieren establecer **cardinalidades mínimas y máximas** en ciertas relaciones. Por ejemplo:

- Un **cliente puede realizar varias compras**, pero cada compra debe estar asociada a un único cliente.
- Un **producto debe pertenecer obligatoriamente a una categoría**, pero una categoría puede contener múltiples productos.
- Un **proveedor puede suministrar varios productos**, aunque cada producto debe tener al menos un proveedor registrado.

Entidades Débiles

En el sistema se identificaron entidades cuya existencia depende directamente de otra entidad principal.

Un ejemplo claro es el **detalle de una venta**, ya que los productos vendidos solo pueden registrarse dentro del contexto de una venta previamente creada. Esto significa que el registro de cada producto vendido **no puede existir de manera independiente**, sino que depende de la existencia de una venta específica.

Por lo tanto, esta estructura se puede modelar como una **entidad débil**, cuya identificación depende de la entidad fuerte **Venta**.

Jerarquías de Generalización / Especialización

El sistema maneja distintos tipos de usuarios que interactúan con la base de datos. En particular, se identifican dos tipos principales:

- **Administradores**, quienes poseen privilegios superiores para gestionar el sistema.
- **Empleados**, quienes realizan operaciones cotidianas como el registro de ventas.

Ambos comparten ciertos atributos generales, como **nombre, contraseña, teléfono y dirección**, pero también presentan diferencias en sus responsabilidades y en la información que se registra de cada uno.

Por esta razón, resulta apropiado modelar esta estructura mediante una **jerarquía de especialización**, donde la entidad **Usuario** actúe como supertipo y las entidades **Administrador** y **Empleado** como subtipos.

Relaciones de Orden Superior

Se identificó que algunas operaciones del sistema involucran la interacción de varias entidades al mismo tiempo. Por ejemplo, el registro de una venta implica la participación de:

- Un **usuario** que registra la operación.
- Un **cliente** que realiza la compra.
- Uno o varios **productos** que forman parte de la venta.

Aunque estas relaciones pueden representarse mediante entidades intermedias, el análisis conceptual permite identificar que se trata de una interacción compleja entre múltiples elementos del sistema.

Ejercicio 2: Reglas de Transformación Básicas

2.1 Transformación de Entidades Fuertes

Regla 1: Toda entidad fuerte se transforma en una tabla.

Para cada entidad fuerte de su modelo:

1. Cree una tabla con el nombre de la entidad

Programa académico / Plan de estudios

2. Transforme cada atributo simple en una columna de la tabla

3. Defina el identificador de la entidad como clave primaria (PK)

4. Especifique el tipo de dato apropiado para cada columna

5. Indique las restricciones de integridad (NOT NULL, UNIQUE, etc.)

Formato esperado:

Tabla: Nombre de la entidad

- atributo1 (PK): tipo_dato [restricciones]

- atributo2: tipo_dato [restricciones]

- atributoN: tipo_dato [restricciones]

Aplique esta regla a todas las entidades fuertes de su modelo.

2.2 Transformación de Atributos Compuestos

Regla 2: Los atributos compuestos se descomponen en sus atributos simples componentes.

Para cada atributo compuesto:

1. Identifique todos los componentes simples

2. Cree una columna para cada componente en la tabla

3. Elimine el atributo compuesto padre

2.3 Transformación de Atributos Multivaluados

Regla 3: Los atributos multivaluados se convierten en una tabla separada.

Para cada atributo multivaluado:

1. Cree una nueva tabla con el nombre del atributo

2. Incluya la clave primaria de la entidad original como clave foránea (FK)

3. Agregue una columna para el valor del atributo multivaluado

4. La clave primaria de la nueva tabla será la combinación de la FK y el atributo multivaluado

2.1 Cardinalidades Mínima y Máxima

A continuación se presentan las relaciones con sus respectivas cardinalidades.

Entidad	**Verbo**	**Entidad**
Cliente	realiza	Venta
Cardinalidad		
Cliente (0, N) — realiza — Venta (1,1)		
Un cliente puede realizar **cero, una o varias compras** dentro de la tienda, ya que no todos los clientes registrados necesariamente han efectuado una compra. Sin embargo, **cada venta debe estar asociada obligatoriamente a un único cliente**, ya que es necesario identificar quién realizó la compra, especialmente en casos donde se requiera facturación.		

Entidad	**Verbo**	**Entidad**
Usuario	registra	Venta
Cardinalidad		
Usuario (0, N) — registra — Venta (1,1)		
Un usuario del sistema (empleado o administrador) puede registrar **muchas ventas a lo largo del tiempo**, pero también es posible que un usuario recién registrado aún no haya realizado ninguna operación. Por otro lado, **cada venta debe ser registrada por un único usuario**, ya que el sistema necesita identificar qué persona realizó la operación dentro del sistema.		

Entidad	**Verbo**	**Entidad**
Categoría	clasifica	Producto
Cardinalidad		
Categoría (1, N) — clasifica — Producto (1,1)		

Cada producto debe pertenecer **obligatoriamente a una sola categoría**, ya que esto permite organizar el inventario y facilitar la búsqueda de artículos. A su vez, una categoría puede contener **uno o varios productos**, dependiendo del tipo de mercancía que agrupe

Entidad	Verbo	Entidad
Proveedor	suministra	Producto
Cardinalidad		
Proveedor (1, N) — suministra — Producto (1,1)		
Un proveedor puede suministrar **varios productos diferentes a la tienda**. Sin embargo, cada producto debe estar asociado **al menos a un proveedor**, ya que se requiere identificar quién es responsable de abastecer dicho producto.		

2.2 Entidades Débiles

Dentro del modelo se identifica una entidad cuya existencia depende directamente de otra entidad principal. Esta entidad se denominan **entidad débil**, ya que no puede existir de manera independiente dentro del sistema.

Para **Detalle_Venta.**

Entidad fuerte asociada	Tipo de dependencia
Venta	Dependencia de identificación.

La entidad Detalle_Venta se utiliza para representar cada uno de los productos incluidos en una venta.

Cada registro de Detalle_Venta **no puede existir sin una venta asociada**, ya que representa únicamente un componente de la transacción. Además, para identificar un registro de esta entidad es necesario combinar la **clave de la venta con la clave del producto**, lo que evidencia su dependencia de identificación.

2.3 Herencia: Subtipos y Supertipos

El sistema contempla diferentes tipos de usuarios que interactúan con la base de datos. Aunque comparten ciertas características generales, cada tipo de usuario posee atributos y funciones específicas dentro del sistema.

Por esta razón se implementa una **jerarquía de generalización/especialización**.

Supertipo: Usuario

La entidad **Usuario** representa a cualquier persona que puede acceder al sistema. Entre los atributos generales que comparten todos los usuarios se encuentran:

- nombre
- contraseña
- número telefónico
- dirección
- correo electrónico

Este supertipo agrupa las características comunes que comparten los diferentes tipos de usuarios del sistema.

Subtipo 1: Empleado

El subtipo **Empleado** representa a los trabajadores de la tienda encargados de realizar las operaciones diarias del sistema, principalmente el registro de ventas.

Atributos:

- número de empleado

- horario de entrada
- horario de salida
- tipo de jornada (3 o 5 días)
- fecha de registro
- estado (activo o inactivo)

Subtipo 2: Administrador

El subtipo **Administrador** representa a los usuarios responsables de supervisar el funcionamiento del sistema y gestionar la información de la base de datos.

Atributos:

- nivel de acceso
- fecha de creación de la cuenta
- permisos administrativos

Tipo de Especialización

Tipo: Disjunta y Total

Justificación:

- **Disjunta**, porque un usuario del sistema solo puede pertenecer a uno de los dos subtipos: **empleado o administrador**, pero no a ambos simultáneamente.
- **Total**, ya que **todo usuario registrado en el sistema debe pertenecer obligatoriamente a uno de estos dos tipos**, lo que asegura que cada cuenta tenga un rol claramente definido dentro del sistema.

2.4 Otros Conceptos

Además de los elementos anteriores, el modelo puede incorporar otros conceptos del modelo EER para representar mejor la información.

Atributos compuestos

Algunos atributos pueden descomponerse en elementos más específicos. Por ejemplo:

Dirección

Puede dividirse en:

- calle
- número
- colonia
- ciudad
- código postal

Esto permite un almacenamiento más estructurado de la información.

Atributos Multivaluados

En ciertos casos, una entidad puede tener más de un valor para un mismo atributo.

Ejemplo:

Un proveedor podría tener **más de un número telefónico**, lo que podría representarse como un atributo multivaluado dentro del modelo

Ejercicio 3: Transformación de Relaciones

3.1 Relaciones Uno a Muchos (1:N)

Regla 4: En las relaciones 1:N, la clave primaria de la entidad con cardinalidad (1) se propaga como clave

foránea a la tabla de la entidad con cardinalidad (N).

Para cada relación 1:N:

1. Identifique la entidad del lado "uno" y del lado "muchos"

2. Agregue la clave primaria de la entidad "uno" como columna en la tabla de la entidad "muchos"

3. Declare esta columna como clave foránea (FK)

4. Determine si la FK debe ser NOT NULL según la cardinalidad mínima

Consideraciones de cardinalidad:

• Si la cardinalidad mínima del lado N es (1,N), la FK es NOT NULL

• Si la cardinalidad mínima del lado N es (0,N), la FK acepta NULL

Documente todas las relaciones 1:N de su modelo con sus tablas resultantes.

3.2 Relaciones Muchos a Muchos (N:M)

Regla 5: Toda relación N:M se convierte en una tabla independiente (tabla asociativa o de unión).

Para cada relación N:M:

1. Cree una nueva tabla con un nombre descriptivo (usualmente el nombre de la relación)

2. Incluya las claves primarias de ambas entidades como claves foráneas

3. La clave primaria de la nueva tabla es la combinación de ambas FK

4. Si la relación tiene atributos propios, agréguelos como columnas adicionales

Consideraciones especiales para cardinalidades específicas:

• Si ambas entidades tienen cardinalidad (0,1) o N a M: se crea tabla asociativa

• Si una entidad tiene cardinalidad (0,1) y la otra (1,1): se suele pasar la clave primaria de (1,1) a la de

(0,1)

• Si ambas tienen cardinalidad (1,1): se pasa la clave de cualquiera de ellas a la otra

3.3 Relaciones Uno a Uno (1:1)

Regla 6: En las relaciones 1:1 existen tres estrategias posibles:

Opción A: Si una entidad tiene cardinalidad mínima (0,1) y la otra (1,1), propague la clave de la entidad (1,1) a

la entidad (0,1) como FK UNIQUE.

Opción B: Si ambas tienen cardinalidad (1,1), propague la clave de cualquiera a la otra como FK UNIQUE NOT

NULL.

Opción C: Fusionar ambas entidades en una sola tabla (solo si tiene sentido semánticamente).

Para cada relación 1:1:

1. Analice las cardinalidades mínimas y máximas

2. Elija la estrategia más apropiada

3. Justifique su decisión basándose en las reglas de negocio

3.4 Relaciones Ternarias y de Orden Superior

Regla 7: Las relaciones de orden superior (ternarias, cuaternarias, etc.) se transforman en una tabla

independiente.

Para cada relación de orden superior:

1. Cree una tabla con el nombre de la relación

2. Incluya las claves primarias de todas las entidades participantes como FK

3. La clave primaria es la combinación de todas las FK (o un subconjunto según el caso)

4. Agregue los atributos propios de la relación

3.1 Diagrama en Notación Peter Chen

De manera análoga, se recurre al uso de softwere draw.io que permite modelar el siguiente caso.

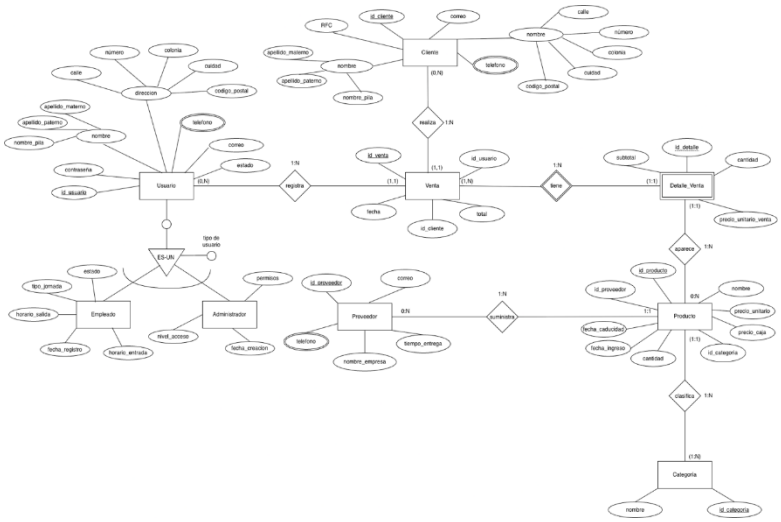


Figura 3.1 Modelo Entidad Relacion Extendido.

3.2 Diagrama en Notación Crow's Feet

Para el siguiente diagrama se hizo el uso de dbdiagram.io.

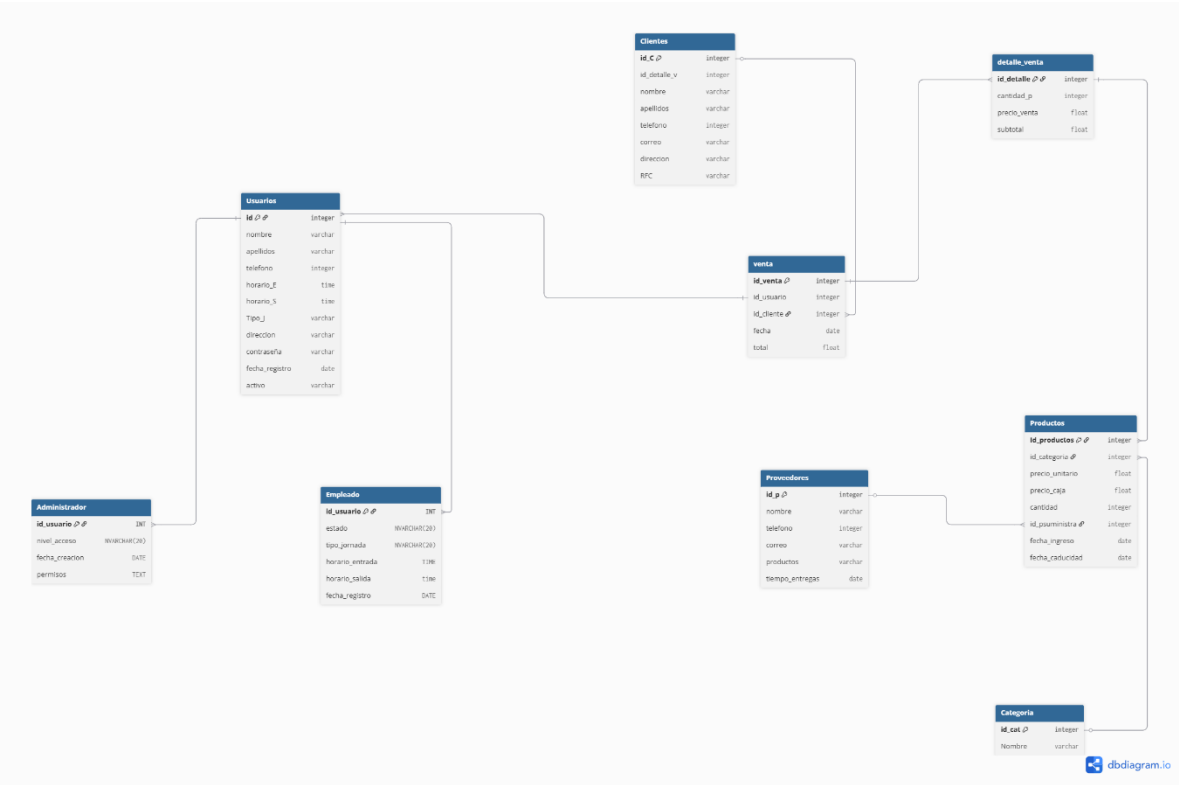


Figura 3.2 Modelo relacional

Ejercicio 4: Transformación de Entidades Débiles

4.1 Entidades Débiles con Dependencia de Identificación

Regla 8: Las entidades débiles se transforman en tablas que incluyen la clave primaria de su entidad propietaria

como parte de su propia clave primaria.

Para cada entidad débil:

1. Cree una tabla para la entidad débil

2. Incluya todos sus atributos propios

3. Agregue la clave primaria de la entidad propietaria como FK

4. La clave primaria compuesta incluye: FK de la entidad propietaria + discriminador de la entidad débil

5. La FK debe ser NOT NULL y con restricción ON DELETE CASCADE

4.2 Entidades Débiles con Dependencia de Existencia

Si una entidad es débil solo por dependencia de existencia (pero tiene su propio identificador único):

1. Cree la tabla normalmente con su propia PK

2. Agregue la FK hacia la entidad propietaria con NOT NULL

3. Configure ON DELETE CASCADE o RESTRICT según las reglas de negocio

4.1 Explicación Detallada del Diseño

En primer lugar, se identifica la entidad Detalle_Venta como una entidad débil, ya que no puede existir de forma independiente dentro del sistema. Su existencia depende directamente de la entidad Venta, debido a que representa los productos incluidos dentro de una transacción específica. Cada registro de detalle requiere estar asociado a una venta previamente creada, lo que implica una dependencia de identificación. Además, su estructura incluye atributos como cantidad, precio unitario y subtotal, los cuales solo tienen sentido en el contexto de una venta, reforzando su naturaleza dependiente.

Por otro lado, el modelo implementa una jerarquía de generalización/especialización a partir de la entidad Usuario, la cual se divide en los subtipos Empleado y Administrador. Esta especialización es de tipo disjunta y total, ya que un usuario únicamente puede pertenecer a uno de los subtipos y todos los usuarios deben estar clasificados en alguno de ellos. Esta decisión permite organizar los atributos comunes en una sola entidad, evitando redundancias, y al mismo tiempo separar los atributos específicos de cada tipo de usuario, como los horarios en el caso de los empleados o los permisos en el caso de los administradores, lo que aporta claridad y orden al modelo.

Asimismo, se establecen cardinalidades mínimas y máximas en las relaciones con el objetivo de reflejar con mayor precisión las reglas del negocio. Por ejemplo, un cliente puede realizar cero o múltiples ventas, pero cada venta debe estar asociada obligatoriamente a un único cliente. De igual manera, una venta debe contener al menos un detalle, lo que asegura que no existan transacciones vacías. Estas restricciones permiten mantener la consistencia de los datos y evitan la generación de información incompleta o inválida dentro del sistema.

Finalmente, al comparar las notaciones utilizadas, se observa que la notación de Peter Chen resulta más adecuada para la representación conceptual del modelo, ya que permite visualizar elementos avanzados como entidades débiles, atributos compuestos y jerarquías de especialización. En contraste, la notación de pata de cuervo facilita la transición hacia el modelo relacional, al representar de forma clara las claves primarias, foráneas y las relaciones entre tablas, lo que la hace más práctica para la implementación en una base de datos. En conjunto, ambas notaciones se complementan y permiten una comprensión integral del sistema desde el nivel conceptual hasta el nivel lógico.

Ejercicio 5: Transformación de Jerarquías de Generalización/Especialización

5.1 Estrategias de Transformación de Herencia

Existen tres estrategias principales para transformar jerarquías ISA:

Estrategia A: Múltiples Tablas con Relaciones 1:1

• Crear una tabla para el supertipo con atributos comunes

• Crear una tabla para cada subtipo con atributos específicos

• La PK del subtipo es FK que referencia al supertipo

Ventajas: Normalización óptima, no hay redundancia ni valores NULL

Desventajas: Requiere JOINs para consultas completas

Cuando usar: Cuando las consultas frecuentemente necesitan solo atributos comunes

Estrategia B: Tabla Única con Todos los Atributos

• Crear una sola tabla con todos los atributos del supertipo y subtipos

• Agregar un atributo discriminador (tipo) para identificar el subtipo

• Los atributos específicos de subtipos aceptan NULL

Ventajas: Consultas simples sin JOINs, buen rendimiento

Desventajas: Muchos valores NULL si los subtipos tienen muchos atributos específicos

Cuando usar: Cuando los subtipos tienen pocos atributos específicos y las consultas frecuentemente necesitan

todos los datos

Estrategia C: Múltiples Tablas Independientes (Sin tabla de supertipo)

• Crear una tabla para cada subtipo con todos sus atributos (comunes + específicos)

• No crear tabla para el supertipo

Ventajas: No requiere JOINs, todas las consultas son directas

Desventajas: Redundancia de atributos comunes, dificulta mantener restricciones generales

Cuando usar: Solo en jerarquías totales y disjuntas donde las instancias siempre pertenecen a exactamente

un subtipo

5.2 Aplicación a Su Modelo

Para cada jerarquía de generalización/especialización en su modelo:

1. Analice las restricciones:

o ¿Es disjunta o solapada?

o ¿Es total o parcial?

2. Elija la estrategia más apropiada:

o Justifique su elección basándose en:

▪ Características de la jerarquía

▪ Patrones de consulta esperados

▪ Redundancia vs. complejidad de consultas

3. Implemente la transformación:

o Diseñe las tablas resultantes

o Especifique todas las PK, FK y restricciones

o Documente el atributo discriminador si aplica

5.1 Comparación con el Modelo Básico

El modelo E-R original presenta varias limitaciones en comparación con el modelo EER actual, principalmente en cuanto a la capacidad de representar con precisión la realidad del sistema. En el modelo básico, la entidad **Usuario** concentra todos los atributos, incluyendo aquellos que solo aplican a ciertos tipos de usuarios, como horarios o niveles de acceso, lo que genera redundancia y falta de claridad. Además, atributos como la dirección se manejan de forma simple, sin descomponerse en elementos como calle, número o ciudad, lo que limita la posibilidad de realizar consultas más específicas. Asimismo, las cardinalidades no reflejan completamente las restricciones de participación mínima y máxima, lo que puede provocar ambigüedad en las relaciones entre entidades.

En contraste, el modelo EER introduce conceptos avanzados que mejoran significativamente la representación del sistema. La especialización permite dividir la entidad Usuario en subtipos como Empleado y Administrador, asignando a cada uno

atributos específicos y evitando inconsistencias. También se incorporan atributos compuestos, como la dirección, y multivaluados, como el teléfono, lo que permite modelar de forma más realista la información. Además, las relaciones están mejor definidas, con cardinalidades completas que especifican claramente las restricciones, y se logra una mayor precisión semántica en la interacción entre entidades como Cliente, Venta, Producto y Detalle_Venta.

Gracias a estas mejoras, el modelo EER permite representar aspectos del problema que no podían modelarse en el enfoque básico, como la diferenciación de roles de usuario, la estructura detallada de atributos complejos, la posibilidad de múltiples valores en ciertos atributos y la correcta definición de dependencias entre entidades. Esto se traduce en un modelo más flexible, preciso y alineado con los requerimientos reales del sistema.

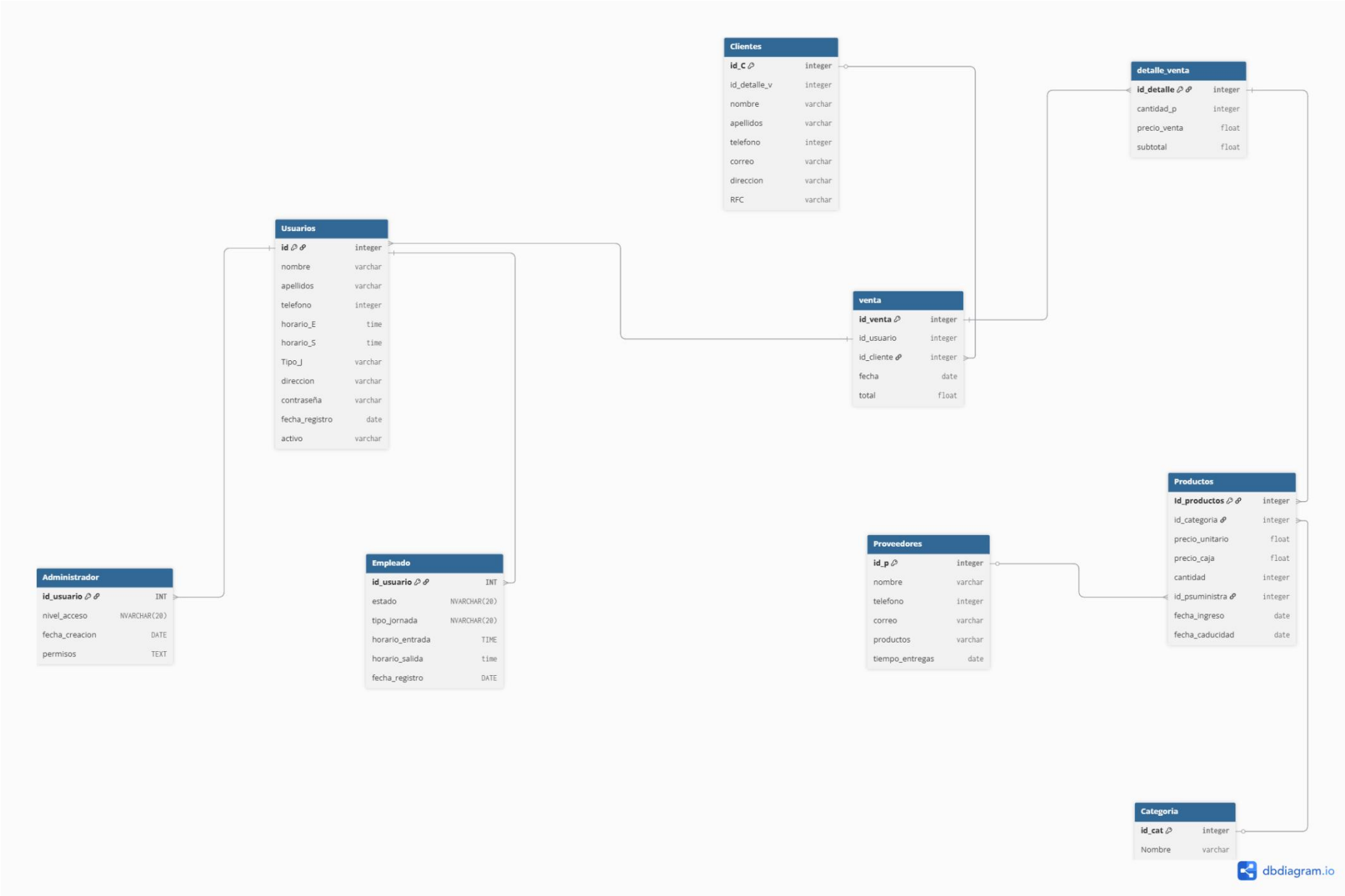
5.2 Lecciones Aprendidas

Durante el proceso de diseño del modelo se presentaron diversas dificultades, especialmente al momento de aplicar conceptos como la especialización y cardinalidad. Determinar cuándo es necesario dividir una entidad en subtipos requiere un mucho análisis, mientras que establecer correctamente las restricciones de participación implica entender a detalle las reglas del negocio. Además, el uso de atributos compuestos y multivaluados aumenta la complejidad del modelo, lo que puede dificultar su construcción si no se tiene una base sólida en modelado conceptual.

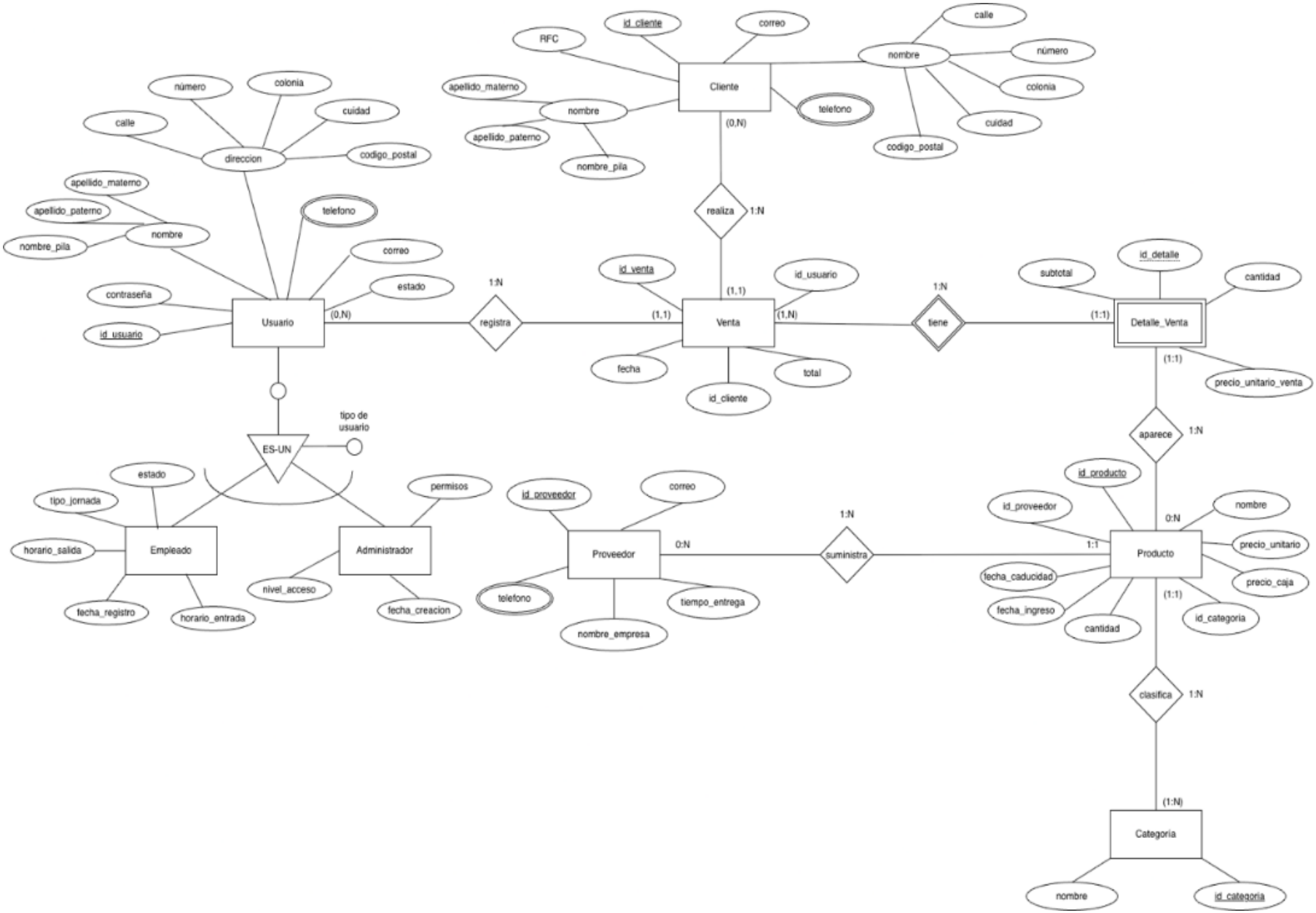
En cuanto a la expresividad, el modelo EER resulta significativamente más potente que el modelo E-R básico, ya que permite representar con mayor detalle la estructura y comportamiento del sistema. Mientras que el modelo E-R ofrece una visión general, el EER incorpora elementos que enriquecen la semántica, como jerarquías,

atributos complejos y restricciones más precisas, lo que facilita una mejor comprensión del sistema.

Finalmente, se reafirma la importancia del modelo conceptual en el desarrollo de sistemas, ya que constituye la base sobre la cual se construye la base de datos. Un buen modelo conceptual permite reducir errores, mejorar la organización de la información y facilitar la implementación de consultas complejas. Además, contribuye a la escalabilidad del sistema, ya que un diseño bien estructurado puede adaptarse con mayor facilidad a futuros cambios o ampliaciones, garantizando la consistencia y calidad de los datos a lo largo del tiempo.


Clientes
id_C integer
id_detalle_v integer
nombre varchar
apellidos varchar
telefono integer
correo varchar
direccion varchar
RFC varchar
detalle_venta
id_detalle integer
cantidad_p integer
precio_venta float
subtotal float
Usuarios
id integer
nombre varchar
apellidos varchar
telefono integer
horario_E time
horario_S time
Tipo_J varchar
direccion varchar
contraseña varchar
fecha_registro date
activo varchar
venta
id_venta integer
id_usuario integer
id_cliente integer
fecha date
total float
Productos
Id_productos integer
id_categoria integer
precio_unitario float
precio_caja float
cantidad integer
id_psuministra integer
fecha_ingreso date
fecha_caducidad date
Proveedores
id_p integer
nombre varchar
telefono integer
correo varchar
productos varchar
tiempo_entregas date
Administrador
id_usuario INT
nivel_acceso NVARCHAR(20)
fecha_creacion DATE
permisos TEXT
Empleado
id_usuario INT
estado NVARCHAR(20)
tipo_jornada NVARCHAR(20)
horario_entrada TIME
horario_salida time
fecha_registro DATE
Categoria
id_cat integer
Nombre varchar
dbdiagram.io

ANEXO


Cliente
id_cliente
correo
RFC
nombre
calle
número
colonia
cuidad
codigo_postal
apellido_materno
apellido_paterno
nombre_pila
telefono
(0,N)
realiza
1:N
Usuario
contraseña
id_usuario
correo
estado
telefono
direccion
número
colonia
calle
cuidad
codigo_postal
apellido_materno
apellido_paterno
nombre
nombre_pila
(0,N)
registra
1:N
Venta
id_venta
id_usuario
fecha
total
id_cliente
(1,1)
(1,N)
tiene
1:N
Detalle_Venta
id_detalle
subtotal
cantidad
precio_unitario_venta
(1:1)
aparece
1:N
ES-UN
tipo de usuario
Empleado
estado
tipo_jornada
horario_salida
fecha_registro
horario_entrada
Administrador
permisos
nivel_acceso
fecha_creacion
Proveedor
id_proveedor
correo
telefono
nombre_empresa
tiempo_entrega
0:N
suministra
1:N
Producto
id_producto
id_proveedor
nombre
precio_unitario
precio_caja
fecha_caducidad
fecha_ingreso
cantidad
id_categoria
1:1
0:N
clasifica
1:N
Categoria
nombre
id_categoria
(1:N)

Ejercicio 6: Esquema Relacional Completo

6.1 Consolidación del Modelo Relacional

Presente el esquema relacional completo resultante de todas las transformaciones. Para cada tabla incluya:

1. Nombre de la tabla

- Usuarios
- Proveedores
- Clientes
- Categoria
- Productos
- Venta
- Detalle_venta
- Empleado
- Administrador

2. Lista de atributos con tipos de datos

- **Usuarios:** id (INTEGER), nombre (VARCHAR(100)), apellidos (VARCHAR(100)), telefono (VARCHAR(20)), horario_E (TIME), horario_S (TIME), Tipo_J (VARCHAR(50)), direccion (VARCHAR(200)), contraseña (VARCHAR(100)), fecha_registro (DATE), activo (VARCHAR(10)).
- **Proveedores:** id_p (INTEGER), nombre (VARCHAR(100)), telefono (VARCHAR(20)), correo (VARCHAR(100)), productos (VARCHAR(200)), tiempo_entregas (DATE).

- **Clientes:** id_C (INTEGER), nombre (VARCHAR(100)), apellidos (VARCHAR(100)), telefono (VARCHAR(20)), correo (VARCHAR(100)), direccion (VARCHAR(200)), RFC (VARCHAR(20)).
- **Categoria:** id_cat (INTEGER), nombre (VARCHAR(100)).
- **Productos:** Id_productos (INTEGER), id_categoria (INTEGER), precio_unitario (FLOAT), precio_caja (FLOAT), cantidad (INTEGER), id_psuministra (INTEGER), fecha_ingreso (DATE), fecha_caducidad (DATE).
- **Venta:** id_venta (INTEGER), id_usuario (INTEGER), id_cliente (INTEGER), fecha (DATE), total (FLOAT).
- **Detalle_venta:** id_detalle (INTEGER), id_venta (INTEGER), id_producto (INTEGER), cantidad_p (INTEGER), precio_venta (FLOAT), subtotal (FLOAT).
- **Empleado:** id_usuario (INTEGER), estado (NVARCHAR(20)), tipo_jornada (NVARCHAR(20)), horario_entrada (TIME), horario_salida (TIME), fecha_registro (DATE).
- **Administrador:** id_usuario (INTEGER), nivel_acceso (NVARCHAR(20)), fecha_creacion (DATE), permisos (TEXT).

3. Clave primaria (PK): Claramente marcada

- Usuarios: id
- Proveedores: id_p
- Clientes: id_C
- Categoria: id_cat
- Productos: Id_productos
- Venta: id_venta

- Detalle_venta: id_detalle
- Empleado: id_usuario
- **Administrador:** id_usuario

4. Claves foráneas (FK): Con referencia a la tabla padre

- **Usuarios, Proveedores, Clientes, Categoria:** No tienen (son tablas independientes o catálogos).
- **Productos:** id_categoria (referencia a Categoria)
- id_psuministra (referencia a Proveedores)
- **Venta:** * id_usuario (referencia a Usuarios)
- id_cliente (referencia a Clientes)
- **Detalle_venta:** * id_venta (referencia a Venta)
- id_producto (referencia a Productos)
- **Empleado:** * id_usuario (referencia a Usuarios)
- **Administrador:** * id_usuario (referencia a Usuarios)

5. Restricciones de integridad:

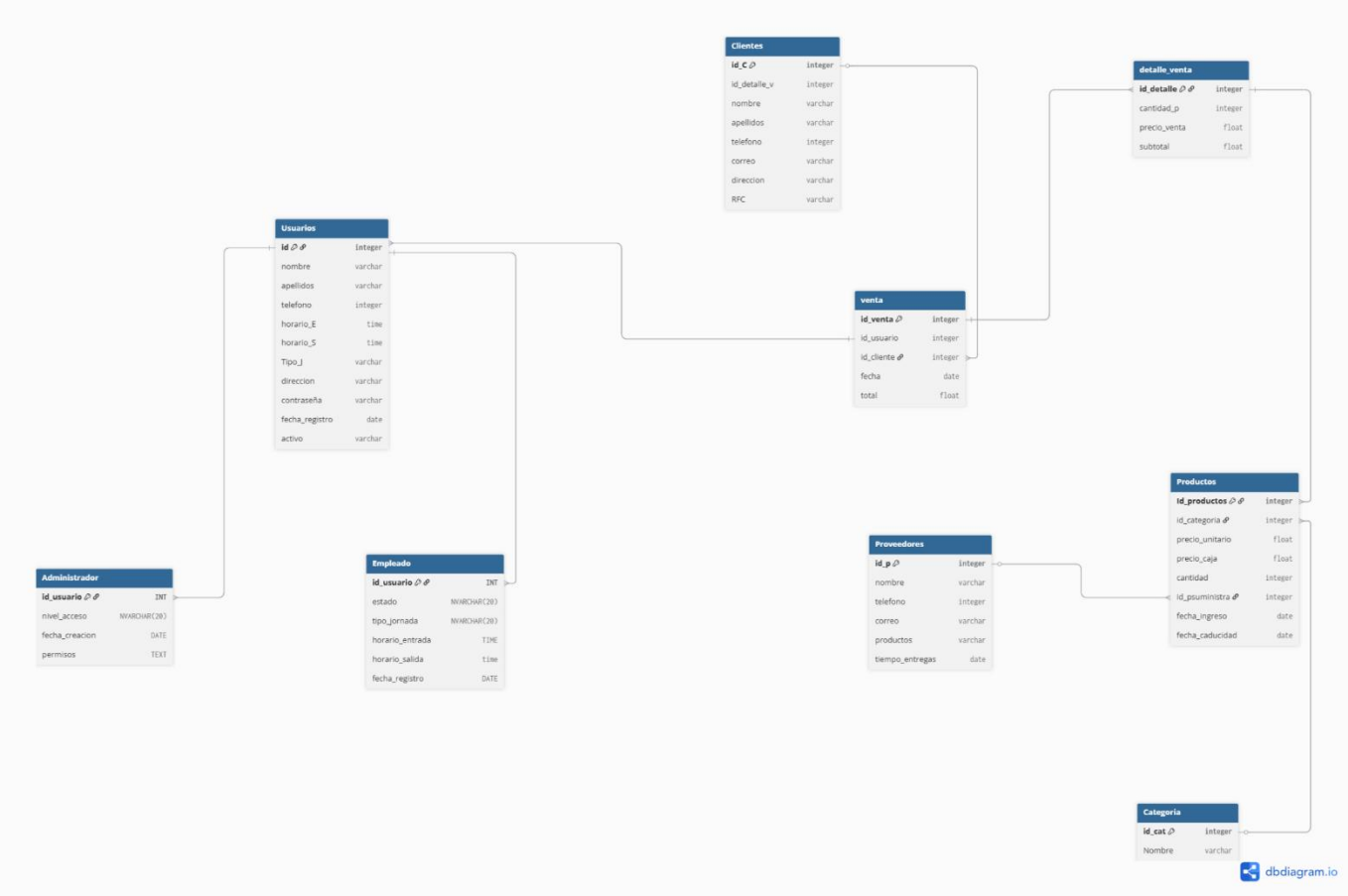
- ✓ **Usuarios: NOT NULL** (id, nombre, apellidos, contraseña); **DEFAULT** (fecha_registro, activo='Activo'); **CHECK** (activo IN ('Activo', 'Inactivo')).
- ✓ **Proveedores: NOT NULL** (id_p, nombre); **UNIQUE** (correo).
- ✓ **Clientes: NOT NULL** (id_C, nombre, apellidos); **UNIQUE** (correo, RFC).
- ✓ **Categoria: NOT NULL** (id_cat, nombre); **UNIQUE** (nombre).

- ✓ **Productos: NOT NULL** (Id_productos, id_categoria, precio_unitario, cantidad); **DEFAULT** (cantidad=0, fecha_ingreso); **CHECK** (precio_unitario >= 0, precio_caja >= 0, cantidad >= 0).
- ✓ **Venta: NOT NULL** (id_venta, id_usuario, id_cliente, total); **DEFAULT** (fecha, total=0); **CHECK** (total >= 0).
- ✓ **Detalle_venta: NOT NULL** (id_detalle, id_venta, id_producto, cantidad_p, precio_venta, subtotal); **CHECK** (cantidad_p > 0, precio_venta >= 0, subtotal >= 0).
- ✓ **Empleado: NOT NULL** (id_usuario); **DEFAULT** (estado='Activo', fecha_registro); **CHECK** (estado IN ('Activo', 'Inactivo', 'Baja')).
- ✓ **Administrador: NOT NULL** (id_usuario); **DEFAULT** (nivel_acceso='Total', fecha_creacion).

6. Acciones referenciales: ON DELETE / ON UPDATE

- Usuarios, Proveedores, Clientes, Categoria: *No aplican.*
- Productos: * FK id_categoria: ON DELETE RESTRICT / ON UPDATE CASCADE
- FK id_psuministra: ON DELETE SET NULL / ON UPDATE CASCADE
- Venta: * FK id_usuario: ON DELETE CASCADE / ON UPDATE CASCADE
- FK id_cliente: ON DELETE CASCADE / ON UPDATE CASCADE
- Detalle_venta: * FK id_venta: ON DELETE CASCADE / ON UPDATE CASCADE
- FK id_producto: ON DELETE RESTRICT / ON UPDATE CASCADE
- Empleado: * FK id_usuario: ON DELETE CASCADE / ON UPDATE CASCADE
- Administrador: * FK id_usuario: ON DELETE CASCADE / ON UPDATE CASCADE

6.2 Diagrama del Modelo Relacional



6.3 Diccionario de Datos

Tabla	Columna	Tipo	Restricciones
Usuarios	id	INTEGER	PK
	nombre	VARCHAR(100)	
	apellidos	VARCHAR(100)	
	telefono	INTEGER	
	horario_E	TIME	
	horario_S	TIME	
	Tipo_J	VARCHAR(50)	
	direccion	VARCHAR(200)	
	contraseña	VARCHAR(100)	
	fecha_registro	DATE	
	activo	VARCHAR(10)	
Proveedores	id_p	INTEGER	PK
	nombre	VARCHAR(100)	
	telefono	INTEGER	
	correo	VARCHAR(100)	
	productos	VARCHAR(200)	
	tiempo_entregas	DATE	
Clientes	id_C	INTEGER	PK

	nombre	VARCHAR(100)	
	apellidos	VARCHAR(100)	
	telefono	INTEGER	
	correo	VARCHAR(100)	
	dirección	VARCHAR(200)	
	RFC	VARCHAR(20)	
Categoria	id_cat	INTEGER	
	nombre	VARCHAR(100)	
Productos	Id_productos	INTEGER	PK
	id_categoria	INTEGER	FK
	precio_unitario	FLOAT	
	precio_caja	FLOAT	
	cantidad	INTEGER	
	id_psuministra	INTEGER	FK
	fecha_ingreso	DATE	
	fecha_caducidad	DATE	
Venta	id_venta	INTEGER	PK
	id_usuario	INTEGER	FK
	id_cliente	INTEGER	FK
	fecha	DATE	
	total	FLOAT	
Detalle_venta	id_detalle	INTEGER	PK
	id_venta	INTEGER	FK
	id_producto	INTEGER	FK
	cantidad_p	INTEGER	
	precio_venta	FLOAT	
	subtotal	FLOAT	
Empleado	id_usuario	INTEGER	PK, FK
	estado	NVARCHAR(20)	
	tipo_jornada	NVARCHAR(20)	
	horario_entrada	TIME	
	horario_salida	TIME	
	fecha_registro	DATE	
Administrador	id_usuario	INTEGER	PK, FK
	nivel_acceso	NVARCHAR(20	
	fecha_creacion	DATE	
	permisos	TEXT	

```sql
CREATE TABLE Usuarios (
   id INTEGER,
   nombre VARCHAR(100),
   apellidos VARCHAR(100),
   telefono INTEGER,
   horario_E TIME,
   horario_S TIME,
```

```sql
   Tipo_J VARCHAR(50),
   direccion VARCHAR(200),
   contraseña VARCHAR(100),
   fecha_registro DATE,
   activo VARCHAR(10),
   CONSTRAINT pk_usuarios PRIMARY KEY (id)
);

CREATE TABLE Proveedores (
   id_p INTEGER,
   nombre VARCHAR(100),
   telefono INTEGER,
   correo VARCHAR(100),
   productos VARCHAR(200),
   tiempo_entregas DATE,
   CONSTRAINT pk_proveedores PRIMARY KEY (id_p)
);

CREATE TABLE Clientes (
   id_C INTEGER,
   nombre VARCHAR(100),
   apellidos VARCHAR(100),
   telefono INTEGER,
   correo VARCHAR(100),
   direccion VARCHAR(200),
   RFC VARCHAR(20),
   CONSTRAINT pk_clientes PRIMARY KEY (id_C)
);

CREATE TABLE Categoria (
   id_cat INTEGER,
   nombre VARCHAR(100),
   CONSTRAINT pk_categoria PRIMARY KEY (id_cat)
);

CREATE TABLE Productos (
   Id_productos INTEGER,
   id_categoria INTEGER,
   precio_unitario FLOAT,
   precio_caja FLOAT,
   cantidad INTEGER,
   id_psuministra INTEGER,
   fecha_ingreso DATE,
   fecha_caducidad DATE,
   CONSTRAINT pk_productos PRIMARY KEY (Id_productos),
```

```sql
   CONSTRAINT fk_productos_categoria FOREIGN KEY (id_categoria)
       REFERENCES Categoria(id_cat)
       ON DELETE RESTRICT ON UPDATE CASCADE,
   CONSTRAINT fk_productos_proveedores FOREIGN KEY (id_psuministra)
       REFERENCES Proveedores(id_p)
       ON DELETE SET NULL ON UPDATE CASCADE
);

CREATE TABLE Venta (
   id_venta INTEGER,
   id_usuario INTEGER,
   id_cliente INTEGER,
   fecha DATE,
   total FLOAT,
   CONSTRAINT pk_venta PRIMARY KEY (id_venta),
   CONSTRAINT fk_venta_usuario FOREIGN KEY (id_usuario)
       REFERENCES Usuarios(id)
       ON DELETE CASCADE ON UPDATE CASCADE,
   CONSTRAINT fk_venta_cliente FOREIGN KEY (id_cliente)
       REFERENCES Clientes(id_C)
       ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Detalle_venta (
   id_detalle INTEGER,
   id_venta INTEGER,
   id_producto INTEGER,
   cantidad_p INTEGER,
   precio_venta FLOAT,
   subtotal FLOAT,
   CONSTRAINT pk_detalle PRIMARY KEY (id_detalle),
   CONSTRAINT fk_detalle_venta FOREIGN KEY (id_venta)
       REFERENCES Venta(id_venta)
       ON DELETE CASCADE ON UPDATE CASCADE,
   CONSTRAINT fk_detalle_producto FOREIGN KEY (id_producto)
       REFERENCES Productos(Id_productos)
       ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE Empleado (
   id_usuario INTEGER,
   estado NVARCHAR(20),
   tipo_jornada NVARCHAR(20),
   horario_entrada TIME,
   horario_salida TIME,
```

```sql
   fecha_registro DATE,
   CONSTRAINT pk_empleado PRIMARY KEY (id_usuario),
   CONSTRAINT fk_empleado_usuario FOREIGN KEY (id_usuario)
       REFERENCES Usuarios(id)
       ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Administrador (
   id_usuario INTEGER,
   nivel_acceso NVARCHAR(20),
   fecha_creacion DATE,
   permisos TEXT,
   CONSTRAINT pk_admin PRIMARY KEY (id_usuario),
   CONSTRAINT fk_admin_usuario FOREIGN KEY (id_usuario)
       REFERENCES Usuarios(id)
       ON DELETE CASCADE ON UPDATE CASCADE
);
```

Código SQL

Ejercicio 7: Validación y Verificación del Modelo

7.1 Preservación de Semántica

Demuestre que su transformación preserva la semántica del modelo original:

1. Todas las entidades están representadas: Verifique que no se perdió ninguna entidad

- Ninguna entidad del diagrama se perdió. Las entidades fuertes (Usuario, Cliente, Proveedor, Categoria, Producto, Venta) se transformaron en tablas principales.
- La entidad asociativa/débil (Detalle_Venta) tiene su propia tabla transaccional.
- Las sub-entidades de la jerarquía (Empleado, Administrador) también están representadas como tablas físicas.

2. Todas las relaciones están implementadas: Confirme que todas las relaciones se transformaron correctamente

- **Cliente realiza Venta (1:N):** Implementada mediante la clave foránea id_cliente en la tabla Venta.
- **Usuario registra Venta (1:N):** Implementada mediante la clave foránea id_usuario en la tabla Venta.
- **Proveedor suministra Producto (1:N):** Implementada mediante la clave foránea id_psuministra en la tabla Productos.
- **Categoria clasifica Producto (1:N):** Implementada mediante la clave foránea id_categoria en la tabla Productos.
- **Venta tiene Detalle_Venta (1:N) y Producto aparece Detalle_Venta (1:N):** Implementadas a través de las llaves foráneas id_venta e id_producto dentro de la tabla Detalle_venta.

3. Cardinalidades respetadas: Muestre cómo las restricciones FK reflejan las cardinalidades originales

- **Relaciones 1:N (Uno a Muchos):** Se respetaron colocando la Clave Foránea (FK) en la tabla que representa el lado "N" (Muchos). Por ejemplo, como un Cliente puede tener N Ventas, la FK id_cliente vive en la tabla Venta.
- **Relación 1:1 (Jerarquía de Usuarios):** La cardinalidad uno a uno de la herencia se garantizó haciendo que la Clave Primaria (PK) de Empleado y Administrador sea exactamente la misma que su Clave Foránea (FK) apuntando a Usuarios(id). Esto impide que un mismo registro hijo se asocie a múltiples padres.

4. Restricciones de integridad: Explique cómo se implementaron las restricciones del modelo EER

- **Integridad de Entidad (PK):** Se garantizó que cada tabla tenga una clave primaria (ej. CONSTRAINT pk_usuarios PRIMARY KEY (id)), asegurando que no haya registros duplicados o nulos en su identificación.
- **Integridad Referencial (FK):** Se utilizaron restricciones FOREIGN KEY con acciones como ON DELETE CASCADE (si se borra una Venta, se borra su Detalle) o ON DELETE RESTRICT (no se puede borrar una Categoría si hay Productos usándola), evitando registros huérfanos.
- **Obligatoriedad (NOT NULL / Participación total):** Aunque en tu código original algunas faltaban, en la consolidación del paso 6.1 agregamos restricciones NOT NULL a las llaves foráneas (ej. id_cliente en Venta) para reflejar que una venta *no puede existir* sin un cliente.

7.2 Comparación de Modelos

Complete una tabla comparativa entre el modelo EER y el modelo relacional:

Aspecto	Modelo EER (Diagrama Original)	Modelo Relacional (Tu Código SQL)	Observaciones
Entidades fuertes	Rectángulos simples (ej. Cliente, Producto, Proveedor).	Tablas independientes con su propia Clave Primaria (PK).	La transformación es directa (1 a 1).
Entidades débiles / Asociativas	Rectángulo con doble línea o rombo conectando entidades transaccionales (Detalle_Venta).	Tabla dependiente cuya existencia requiere de llaves foráneas (FK) hacia Venta y Productos.	El modelo relacional exige que se convierta en tabla física.
Relaciones N:M (Muchos a Muchos)	No hay un rombo N:M explícito, pero	Se resolvió utilizando la tabla	El modelo relacional no soporta N:M

	lógicamente Venta y Producto tienen esta relación.	puente Detalle_venta.	directo; siempre requiere una tabla intermedia.
Jerarquías ISA	Triángulo invertido con la etiqueta ES-UN apuntando a Usuario.	Tres tablas: 1 padre (Usuarios) y 2 hijas (Empleado, Admin). Las hijas usan el id_usuario como PK y FK simultáneamente.	Se eligió el mapeo de "múltiples tablas" para preservar atributos específicos sin dejar valores nulos.
Atributos multivaluados	Óvalos con doble línea (ej. telefono en Cliente, Usuario, Proveedor).	Simplificado: Una sola columna telefono VARCHAR/INTEGER en la tabla principal.	
Atributos compuestos	Óvalo conectado a otros óvalos (ej. direccion -> calle, numero, colonia).	Simplificado: Una sola columna de texto libre direccion VARCHAR(200).	

Bibliografía

[1] A. Silberschatz, H. F. Korth y S. Sudarshan, *Fundamentos de bases de datos*, 6ª ed. México: McGraw-Hill, 2011.

[2] L. Date, *Introducción a los sistemas de bases de datos*, 8ª ed. México: Pearson Educación, 2004.

ANEXO

[3] Universidad Nacional Autónoma de México, “Modelo entidad-relación,” material de curso, Facultad de Ciencias, UNAM. [En línea]. Disponible en: https://hp.fciencias.unam.mx/~alg/bd/er.pdf